QUANTUM QLi

QUANTUM QLi

TECHNICAL WHITE PAPER

Launch infrastructure / Flywheel operations

Cryptographic identity / Authorization

IMPLEMENTED CAPABILITIES | VERIFIED BOUNDARIES | ROADMAP

VERSION 0.2 - 11 OCTOBER 2026

github.com/QuantumQLi/QLi - quantumqli.app

EDITION NOTES

What this paper covers

Quantum QLi is an inspectable, non-custodial toolkit for Solana token launches, liquidity workflows, identity proofs, and transaction-specific authorization. This edition brings together
the public repository's foundational technical white paper and the more recent application
implementation reviewed on 11 October 2026.

A PRECISE DISTINCTION

A post-quantum proof can be authentic without being enforced by the blockchain. Quantum QLi currently
verifies identity-linked authorization inside the application. The network still enforces its native transaction and
program rules. No deployed QLi post-quantum treasury verifier is claimed.

Evidence labels used throughout

LABEL INTERPRETATION

IMPLEMENTED Application code or cryptographic operation exists; end-user or network acceptance may still
depend on external services.

NETWORK PATH Code builds, signs, broadcasts and confirms real transactions subject to wallet/RPC readiness; not
the same as a demonstrated funded public-mainnet execution.

LOCAL / REFERENCE Executed in the browser or locally, with no automatic public registry, custody, or enforcement.

PLANNED A proposed design milestone, not deployed functionality or a delivery commitment.

The GitHub main branch still presents an earlier white paper (v0.1) and some older planning-only descriptions. The latest project source
includes newer launch execution, Flywheel and Quantum Authorization components. Where source vintages differ, this edition describes
the newer implementation and calls out missing public-chain verification.

Scope: technology, architecture, threat model, current status, research roadmap and operational caveats. This document is not a token
offering, audit certificate, investment recommendation or financial forecast.

Repository: github.com/QuantumQLi/QLi . Product: quantumqli.app

CONTENTS

Inside this edition

A source-aligned explanation of the system as it exists today, the boundaries of its cryptographic
security, and the next conditions required for program-enforced authorization.

What this paper covers 2
Executive summary 5
Problem, design principles and system boundaries 5
Product surface and operational modules 6
Launch Forge: transaction path 7
Quantum Launch: provenance, not chain authority 8
Flywheel Engine: claim, buyback and burn 8
Quantum Identity and hash-based proofs 9
Quantum Authorization v1: signed transaction intent 10
Authorization versus enforcement 11
Standardized post-quantum primitives 12
Wallet lifecycle and transaction integrity 13
Vaults, key recovery and missing custody 13
API, data model and software architecture 14
Roadmap: the on-chain treasury rejection demo 15
Testing, evidence and release criteria 16
Risk disclosures, governance and tokenomics scope 17
Conclusion 17
Glossary and source register 18

READING GUIDE

Chapters 4 - 7 describe currently implemented launch, Flywheel and proof flows. Chapters 8 - 10 distinguish
cryptographic authenticity from protocol enforcement. Chapter 14 defines a concrete on-chain rejection
demonstration that has not yet been delivered.

CHAPTER 01

Executive summary

Quantum QLi combines non-custodial Solana operations with locally generated cryptographic identity and explicit
transaction review. Its ambition is to help creators move from launching an asset toward governing how that asset is
managed, verified, and eventually protected by enforceable authorization policies.

Three advances distinguish the October 2026 implementation from the repository's original v0.1 white paper: a real
SDK-backed Meteora DBC launch transaction path; a wallet-approved Flywheel sequence for creator-fee claims,
buybacks and burns; and Quantum Authorization v1, which attaches one-time identity proofs to exact Flywheel
transaction intents before wallet approval. [R1 - R3, U1 - U3]

What is working, and what remains a boundary

AREA STATE AS OF 11 OCT 2026

Launch Forge SDK-backed DBC config-and-pool construction, preflight checks, wallet-signing, submission and
account re-reads are implemented. No funded public-mainnet launch was independently demonstrated in reviewed evidence.

Flywheel Live Creator fee claim, same-pool SOL buyback and token burn transaction paths are implemented as
separate wallet-approved steps. Scenario fee allocations are not automatic on-chain routing.

Quantum Identity Real local reference WOTS/Merkle proof derivation, signing and verification with 256 one-time
leaves. Local reuse ledger only.

Quantum Authorization v1 Opt-in application policy binding a WOTS/Merkle proof to reviewed transaction context. Browser
verification before signing; bypassable by other clients.

Post-quantum workbench Library-backed browser operations for ML-DSA-65, SLH-DSA-SHA2-128s and ML-KEM-768.

Quantum Vault / verifier No deployed on-chain post-quantum custody verifier; deposit/withdrawal prototypes do not move
protected assets.

CORE THESIS

The immediate product value lies in explicit wallet-authorized operations plus inspectable proof provenance. The
proposed next security milestone is a Solana program that rejects an unauthorized vault action on-chain, even
when called outside the Quantum QLi interface.

CHAPTER 02

Problem, design principles and system boundaries

Token creation, liquidity deployment, fee accrual, treasury actions and security policy are often spread across multiple
interfaces. In that setting, the user can lose track of what was configured, what was cryptographically authenticated,
what was merely simulated, and what was actually settled.

Quantum QLi addresses this with a common review surface and explicit evidence tiers. It does not assume that adding a
hash or proof to an interface makes the underlying asset secure. A valid identity proof demonstrates only what its signed

message, key material and trust context actually establish.

Non-custodial layered architecture

01 INTERFACE / INTENT

Launch Forge . Flywheel . Quantum Authorization

02 LOCAL CRYPTO / POLICY

WOTS/Merkle identity . canonical proofs . client guard

03 WALLET AUTHORIZATION

Independent approval of the reviewed transaction

04 NETWORK / PROGRAMS

Solana and Meteora rule enforcement + account state

- Human authorization: a compatible wallet approves each real transaction; the application does not maintain an unattended signer.
- Immutable intent during review: changes to important parameters invalidate prepared transactions or identity proofs.
- Public verification: committed proof material can be checked without access to the original signing secret.
- Honest failure modes: errors, expired blocks, absent RPC access and unsupported settings stop execution rather than silently downgrade configuration.
- Separation of models and settlement: forecasts, demos and fee-allocation proposals are not chain receipts.

CHAPTER 03

Product surface and operational modules

The application uses a desktop-inspired interface and a set of specialized modules. Not every module has identical security or persistence properties: some execute real cryptographic operations, some read RPC state, some prepare reviewed transactions, and others remain illustrative.

MODULE PRIMARY ROLE EVIDENCE CLASS

Launch Forge Seven-stage asset, curve, fees, migration, quote, first-buy and review workflow. Network path (limited)

Quantum Identity Wallet-bound local reference derivation; registration, challenge proofs, and memo preparation. Local / reference

Proof Inspector Parse and independently verify supported identity and launch proof data. Local verifier

Quantum Opt-in operation-policy settings, proof creation, verification and Application policy Authorization authorization records.

Flywheel Live Claim creator fees; buy back through own eligible DBC pool; burn held tokens. Network path (limited)

MODULE PRIMARY ROLE EVIDENCE CLASS

Flywheel Scenarios Fee allocation sliders, revenue forecasts, digest/proposal generation. Hypothetical model

Portfolio / Activity Read wallet/RPC state where available. Connected-wallet read

Vault Desk / Prototype ML-DSA key, signing, recovery and undeployed WOTS vault Research / local
Quantum Wallets concept.

QLi Operator / Strategy previews and illustrative liquidity datasets; no autonomous Demo / planning
Markets signer.

A separate public API exposes bounded, public cryptographic operations rather than a custodial signing service. The
private-key operations, on-chain vault and globally synchronized identity registry needed for fully enforced
post-quantum custody do not exist in this snapshot. [R1, R8, U1]

CHAPTER 04

Launch Forge: transaction path

Launch Forge begins with a seven-stage configuration wizard. Although a broad set of parameters is presented for
planning, the executable subset deliberately narrows real deployment to a DBC config-and-pool transaction where the
selected properties can be applied precisely.

Four-stage launch execution

BUILD SIMULATE APPROVE CONFIRM

SDK + account checks Frozen message Wallet signatures Chain state re-read

The current live construction uses the Meteora Dynamic Bonding Curve SDK, including createConfigAndPool, against
program dbcij3LWUppWqq96dh6gJWwBifmcGfLSB5D4DuSMaqN. The user's wallet is payer, fee claimer, leftover
receiver and pool creator. It requires wrapped SOL as the quote asset, immutable token authority and DAMM v2 as
migration target. Unsupported first-buy, vesting, fee-scheduler, non-SOL quote, vanity mint and Token-2022 launch
settings block execution rather than being omitted silently. [U1]

Preflight and execution invariants

- RPC genesis hash and invoked program allowlist are checked; the DBC program and quote mint must exist on the
selected network.
- Hosted metadata JSON must agree with the proposed token name and symbol and provide an HTTPS image; metadata
immutability is not guaranteed by ordinary web hosting.
- The reviewed signer set must consist of the wallet payer plus fresh config and mint keys; required signatures are
verified before broadcast.
- Pre-wallet simulation, expiry and signer checks, reviewed message-byte integrity, single-submission handling and
post-confirmation account ownership checks are implemented.

- Mainnet includes a separate risk acknowledgement and a 0.1 SOL application preflight threshold. This is not an
on-chain spending limit or final wallet fee guarantee.

Verification status

The launch path was tested against a local validator with DBC and Metaplex programs cloned from devnet, producing a
confirmed config and pool. Public mainnet/devnet RPC dry runs also checked program, genesis and signer assumptions
but stopped before funding/signing. The reviewed project checklist records no funded public-mainnet transaction, and
no physical Phantom extension/mobile wallet confirmation. These are readiness limitations, not evidence that a funded
mainnet transaction occurred. [U1]

CHAPTER 05

Quantum Launch: provenance, not chain authority

Quantum Launch adds a reference WOTS/Merkle Attestation Seal to the standard DBC launch process. The metadata
JSON must contain a proof that verifies against the open Quantum Identity and the canonical launch data. Public-demo
secrets are refused for real attribution.

The v1 launch digest commits to version, mode, name, symbol, metadata URI, decimals, supply, quote mint, DBC
program and DAMM v2 migration selection. It does not commit to every curve, fee, liquidity, social-link or first-buy
setting. Therefore a verified seal must not be interpreted as endorsement of the complete economic configuration. [R5,
U1]

ATTESTATION MEANING

Successful WOTS/Merkle verification says the specified signed launch fields match the proof and identity root. It
does not make the token immutable, prove token ownership at some later date, guarantee a pool was deployed,
or replace Solana authorization.

Metadata integrity risks

A token metadata URI may point to a mutable HTTPS host. The signed canonical fields can establish consistency at
signing time, but changes to remotely hosted data or omitted configuration fields may fall outside that commitment. A
stronger future design would include immutable/content-addressed metadata, complete economic-field commitment
and independent confirmation of on-chain ownership and state.

CHAPTER 06

Flywheel Engine: claim, buyback and burn

The Flywheel Engine connects three eligible creator-controlled operations to the asset's own Meteora DBC pool. It is a
manual, wallet-approved cycle: no recurring worker, background signer or automatic treasury automation is present. The
live operations are distinct from the adjacent allocation and forecasting models. [U2]

01 CLAIM 02 BUY BACK 03 BURN

Creator trading fees SOL to pool token SPL BurnChecked

Supported live-step rules

STEP PROTOCOL ACTION OPERATIONAL GUARD

Claim SDK creator trading-fee claim for a pool created by On-chain pool/creator read, reviewed account set and
the connected wallet. wallet approval.

Buyback Same-pool SOL swap with an on-chain quote. Minimum-out slippage floor; configurable cap with 10
SOL hard application ceiling; migrated and non-SOL
pools blocked.

Burn SPL Token BurnChecked from user-owned token Token balance constraint; irreversible supply reduction
account. on successful settlement.

Each transaction is simulated, signed independently, broadcast with single-send safeguards and checked after
confirmation. The interface re-reads relevant balances, supply and claimable fee state; only observed confirmations are
labelled "Verified on-chain." Burns reduce supply but do not guarantee price effects or economic returns.

What the Flywheel does not automate

Revenue projections, allocation percentages, proposed buyback shares and SHA-256 proposal digests are local scenario
tools. No contract automatically redistributes creator fees according to these percentages. A proposed allocation does
not create an on-chain accounting obligation. Pool migration to DAMM v2 currently blocks the live same-pool buyback
path pending a separate swap integration.

CHAPTER 07

Quantum Identity and hash-based proofs

Quantum Identity is a reference identity mechanism derived locally after wallet message signing. It binds an address and
selected network to a signed derivation message, then uses HKDF-SHA256 for local expansion. An optional passphrase
path uses NFKC normalization and scrypt (N = 2^15, r = 8, p = 1). Deterministic re-derivation depends on a stable signing
response and identical inputs. [R4]

WOTS/Merkle reference construction

PROPERTY CURRENT REFERENCE PARAMETER

Underlying digest SHA-256

Winternitz base w = 16

Signature chains 67 chains x 32-byte elements

Merkle depth 8, allowing 256 leaves

Raw reference proof 4-byte leaf index + 2,144-byte signature + 256-byte path = 2,404 bytes

Domain separation Versioned proof/launch domains and explicit authorization intent prefix

Registering an identity reserves the first leaf for a binding message; later possession or authorization proofs reserve the
next unused leaf before signing. Verification recomputes the WOTS leaf and follows the Merkle authentication path to
the public root. One-time keys must never be reused: exhausted or corrupted key history is a security failure, not a
condition to silently reset.

Where persistence stops

The consumed-leaf ledger stores indexes and purpose labels in the browser. It is neither globally synchronized nor
atomic across devices; clearing storage, using multiple tabs or losing recovery data can permit unsafe reuse. No
production-grade identity registry or certified XMSS-compatible implementation is claimed. Session secret handling also
cannot guarantee full JavaScript memory erasure.

CHAPTER 08

Quantum Authorization v1: signed transaction

intent
Quantum Authorization extends the reference identity mechanism to a concrete Flywheel transaction prepared for user
review. When policy is enabled for that operation, the application asks the identity holder to sign a canonical, versioned
authorization intent with an unused WOTS leaf. A verifier must accept the proof before the wallet-sign prompt becomes
available. [U3]

Fields committed by the intent

DOMAIN / IDENTITY OPERATION / TRANSACTION

Version and operation type Pool, creator, mint and recipient

Network and genesis hash Amount in base units; maximum spend

Wallet account and identity address/root Slippage and minimum output, as applicable

Issued-at and expiry timestamps SHA-256 of the reviewed message

Random 32-byte nonce Recent blockhash and last valid block height

The intent is canonicalized using a fixed key order and deterministic string values, prefixed with the versioned
authorization domain, hashed using SHA-256 and signed under the reference identity proof domain. The verifier checks
proof kind, domain, root match, exact signed message, current operation fields, timestamps, identity context and
in-session nonce use.

Application-level policy and record lifecycle

- Policy is opt-in, with individual coverage of creator-fee claim, buyback and burn. For buybacks, a threshold may trigger
identity approval at or above the chosen amount.
- Authorization expiry may be set from 30 to 900 seconds; a different wallet, quote, RPC, policy, network, blockhash,
identity or reviewed message invalidates the prior approval.
- Proof is rechecked immediately before wallet signing. Public-demo identities, missing proofs, stale proofs and
tampered/mismatched transaction data are rejected by the QLi application.
- Authorization records distinguish Created, Verified locally, Expired, Invalidated, Rejected and Executed on-chain. The
last status requires the separate wallet transaction to be confirmed and on-chain state to be re-read.
- Public proof export contains verification material, not the private leaf seeds. Records are session-scoped; settings are
browser-local and mutable.

THE HARD BOUNDARY

Quantum Authorization v1 is NOT a Solana program rule. A wallet owner or attacker with the relevant
conventional authority can bypass this additional proof check by calling the network outside Quantum QLi. It is
an application-level usability and review safeguard, not quantum-safe custody.

CHAPTER 09

Authorization versus enforcement

Security architecture must distinguish three statements that can sound similar but have different consequences. (1) "A
signature is valid" is a cryptographic claim about a message and key. (2) "The app refuses an action" is a client-side policy
behavior. (3) "The program rejects an action" is a blockchain settlement rule, independently enforced across clients.

SECURITY QUESTION CURRENT ENFORCER BYPASS / LIMITATION

Who approves the Solana transaction? Wallet signature + native Conventional Solana authorization remains
programs decisive.

Does this WOTS proof match the signed Local verifier Valid proof does not force any on-chain rule.
intent?

Can Flywheel sign without an enabled QLi application gate Another client can bypass application UI
proof? checks.

Does a spend cap bind all transaction QLi client for stated operations Program-level cap not deployed by QLi.
clients?

Can funds leave a post-quantum protected No current QLi verifier/vault No such protected custody path exists.
QLi vault?

Can a treasury reject missing PQ proof Not implemented Requires deployed, tested, auditable program
on-chain? logic.

Why hybrid authorization is the probable near-term route

An on-chain enforcement design could preserve ordinary wallet approval while adding a second, program-verified
cryptographic condition for assets held under a purpose-built vault authority. That is more realistic than suggesting a
local identity proof upgrades every existing Solana account. The new verifier must bind the proof to an exact instruction,
not merely to an account or arbitrary statement.

This is a proposed architecture only. Choice of post-quantum scheme, computational feasibility, signature transport,
chain-time expiry, gas/compute budget, recovery and key rotation require engineering and external security review.

CHAPTER 10

Standardized post-quantum primitives

Quantum QLi includes a browser-local cryptographic workbench built on @noble/post-quantum. These operations are
separate from the reference WOTS identity system and from the account signatures that authorize ordinary Solana
transfers. Algorithm presence is not evidence of certified implementation security or deployed quantum-resistant
custody. [R3]

SCHEME REFERENCE CURRENT ROLE

ML-DSA-65 FIPS 204 Local post-quantum digital-signature operations; public verification API.

SLH-DSA-SHA2-128s FIPS 205 Local stateless hash-based signature operations; public verification API.

ML-KEM-768 FIPS 203 Local encapsulation/decapsulation and public encapsulation API.

Custom WOTS/Merkle Reference, not One-time identity possession, launch and authorization attestations.
FIPS-certified as
implemented

SCHEME REFERENCE CURRENT ROLE

FN-DSA / Falcon Experimental Not a deployed launch certification or custody signer. exploration

Hybrid Ed25519 + ML-DSA Research direction No deployed on-chain enforcement integration.

Interpreting "post-quantum" correctly

"Post-quantum" describes the cryptographic hardness assumptions and algorithm families targeted against certain
quantum attacks. It does not imply that browsers, wallets, RPC nodes, smart contracts, key storage, metadata hosts or
governance are secure. In particular, user funds in ordinary Solana accounts remain governed by the network's normal
authorization model.

CHAPTER 11

Wallet lifecycle and transaction integrity

Real transaction submission uses an explicit separation of preparation, simulation, wallet authorization, broadcast and
confirmation. This makes the difference between a user-approved intent and an observed state transition visible in the
interface.

STATE WHAT IT ACTUALLY MEANS

Built A transaction message and account set were constructed.

Simulated The selected RPC evaluated the candidate transaction. Not network settlement.

Awaiting wallet User has not yet signed; the application cannot presume approval.

Submitted Signed bytes were broadcast; confirmation not yet established.

Confirmed A signature was observed with the configured commitment and account state was re-read.

Failed / expired There is no successful confirmed state to report; a new reviewed transaction may be required.

Safety mechanisms include payer and signer validation, full-genesis RPC checks, byte-for-byte review matching,
expired-blockhash rejection, single-submission handling and post-signature re-verification. A confirmation still does not
establish that the transaction was economically wise or immune to market risk. [U1 - U2]

CHAPTER 12

Vaults, key recovery and missing custody

The Vault Desk exposes browser-local ML-DSA-65 key creation and public fingerprints. Explicit offline seed backup and
restore are provided; local withdrawal proof generation can be demonstrated. However, neither the current vault
screens nor the reference WOTS wallet-chain staging move funds out of an enforceable QLi post-quantum-controlled
vault. [R7, U4]

NO DEPOSITED VAULT ASSETS

The current Quantum Wallet program identifier is unset; vault creation, deposit, withdrawal and independent
post-quantum settlement checks are unavailable. A locally valid withdrawal signature must not be mistaken for
permission granted by a deployed Solana program.

Operational key-management concerns

- Offline recovery of deterministic reference identities depends on exact derivation inputs, signing behavior and an intact
consumed-leaf history.
- The prototype ML-DSA backup seed is sensitive; never paste it into support, cloud chats, public issues or logs.
- Clearing local memory does not prove that browser heap copies, extensions or the host operating system cannot retain
data.
- A future production vault requires formalized key rotation, loss handling, replay storage, multi-party policy and
emergency response - without introducing bypassable administrator backdoors.

CHAPTER 13

API, data model and software architecture

The TypeScript application uses React and TanStack routing with domain service modules for DBC transaction
preparation, local identity, proof checking and explicit wallet adapters. Application session state is not an on-chain
database, and no public private-key signing endpoint is exposed. [R1, R8]

Public cryptographic API

HTTP PATH SUFFIX (under /api/public/v1) BEHAVIOR

GET algorithms, health Advertises cryptographic capabilities and self-tests.

GET health/history Volatile worker history, not an uptime guarantee.

POST signatures/verify Public ML-DSA or SLH-DSA signature verification.

POST kem/encapsulate ML-KEM ciphertext and a fingerprint, not a disclosed shared secret.

501 keys, signatures/sign, kem/decapsulate, usage No server private-key generation/signing/decapsulation or
per-project usage.

Public JSON requests are bounded to 65,536 bytes. Cryptographic public endpoints are intentionally narrower than local
workbench operations. Worker history and some strategy/event records are volatile and not a durable event-indexer,
compliance system or confidential key service.

Data classes and storage

- On-chain: pool/program accounts, mints, transactions, confirmed signatures and balances read through RPC.
- Public cryptographic material: signature proofs, roots, public seeds, authentication paths and canonical signed fields.
- Browser-local: policy preferences, consumed leaf indexes, ephemeral proof records and tab/session state.

- Private local: wallet approvals, WOTS secrets, passphrases and prototype ML-DSA material. The product has no
general-purpose remote secret escrow.

CHAPTER 14

Roadmap: the on-chain treasury rejection demo

The most meaningful next security milestone is not another dashboard: it is an independently repeatable demonstration
that a protected Solana treasury refuses unauthorized movement inside the on-chain program, even if an attacker uses a
different client. No such QLi verifier is deployed at this time.

Proposed acceptance criteria

VAULT VERIFY SETTLE

Program-held authority Exact signed action Atomic CPI + nonce

- Custody boundary: put test assets under a program-derived authority so a generic wallet transfer alone cannot bypass
the vault's policy.
- Canonical authorization: bind the proof to program ID, vault address, network/genesis, mint, recipient, amount,
operation, expiry and unique nonce. Define a stable, auditable serialization.
- On-chain decision: verify approved cryptographic proof and all relevant policy fields before SPL Token cross-program
invocation. Store consumed nonces in chain state, atomically preventing replays.
- Negative tests: missing proof, forged signature, wrong recipient, changed amount, wrong network, expired
authorization, reused nonce and stale/rotated key must all be rejected by the program.
- Independent demonstration: run local-validator tests, deploy a strictly limited devnet test vault, publish program ID,
instruction schema, code, negative transaction logs and externally reproducible steps.
- Production gate: benchmark proof footprint and compute cost, complete independent security review, publish upgrade
authority policy, and only then consider mainnet value-bearing deployment.

PROTOCOL ENGINEERING CONSTRAINT

A reference WOTS proof carries about 2,404 raw bytes before encoding. Solana transaction and compute
constraints mean an on-chain verifier design cannot simply assume it fits into one ordinary instruction. The
verifier, proof transport, scheme selection and replay architecture require measured engineering.

Progression, not guarantees

PHASE MILESTONE STATUS

I Complete current application intent, verifier, policy, proof export and negative tests. Implemented, app-level

II Design program-owned vault, shared instruction schema and on-chain replay state. Planned

III Produce reject/accept tests on local validator and public devnet; publish evidence. Planned

IV External audit, operational recovery policy, governance, then limited mainnet rollout. Conditional / future

CHAPTER 15

Testing, evidence and release criteria

The project includes regression coverage for core Solana transaction boundaries, proof verification and navigation. The
reviewed October 2026 authorization update reports a build with 113 passing tests and six skipped tests, plus
successful typecheck, lint, production build and desktop/mobile route checks. These are developer-run checks, not an
independent security certification. [U3]

Observed versus not yet demonstrated

EVIDENCE WHAT IT SUPPORTS

Local-validator DBC launch Real account creation, signing/confirmation and ownership checks in controlled conditions.

Public RPC dry runs Program availability, genesis and signer assumptions with unfunded address; no funded
broadcast.

Authorization negative unit tests Tampered intent, expiry, nonce replay in session, identity mismatch, policy bounds and leaf
reuse refusal.

Desktop/mobile smoke checks Basic routes render without reported page errors or overflow.

No physical Phantom funded test Cannot claim end-to-end funded wallet workflow is independently demonstrated.

No separate completed security audit Cannot claim audited production treasury or quantum-safe custody.

Release checklist before reliance on value-bearing operations

- Run a fresh funded-wallet transaction with the intended RPC, wallet extension and pool conditions; independently
verify signature and on-chain account state.
- Reproduce failed-path tests for wallet cancellation, changed signed bytes, stale blockhash, wrong genesis, quote
change, double broadcast and confirmation timeout.
- Document the operator's security model, supported networks, exact fee recipients and program authorities.
- Publish cryptographic test vectors and full reproducibility information for identity
and authorization proof formats.

- Perform a dedicated dependency review, external code audit and security disclosure process before custody or
automated treasury controls.

CHAPTER 16

Risk disclosures, governance and tokenomics scope

Quantum QLi is a technical infrastructure project, not a representation that any token appreciates in value or generates
guaranteed yield. Creator trading fees, buybacks and burns are network operations subject to market conditions, price
impact, slippage, liquidity and third-party protocol risk. Burning tokens changes recorded supply; it does not guarantee a
price outcome.

The proposed post-quantum treasury system is not live. An app-enforced proof gate can be bypassed by clients that do
not call it; a standard wallet compromise may still authorize assets outside future program-controlled vaults. Mutable
metadata, browser-local key ledgers, wallet extensions, RPC exposure, protocol upgrades and governance errors remain
meaningful attack surfaces.

Governance / token economics

The available source does not establish a universal QLi token supply, public fee share, emissions schedule, investor
allocation, treasury reserves or protocol-governed revenue split. This paper intentionally invents none. Economic policies
shown in the Flywheel scenario interface are hypotheses until a real on-chain authority, explicit instructions and
independent state inspection demonstrate enforcement.

Licensing and intellectual property

The public repository states Apache License 2.0 for source code and documentation. Quantum QLi names, marks and
original artwork are excluded from that general source-code grant unless separately licensed. Third-party software
remains subject to its own licensing terms. [R9]

CHAPTER 17

Conclusion

Quantum QLi's present architecture is most useful when each layer is interpreted for what it can actually enforce. The
SDK-backed Launch Forge and Flywheel transaction paths give creators explicit, reviewable network operations.
Quantum Identity provides locally generated, independently verifiable reference cryptographic proofs. Quantum
Authorization ties those proofs to precisely reviewed operations - today as an additional application gate, not as a
network permission rule.

The next step is architectural, not cosmetic: demonstrate a program-controlled vault that rejects an unauthorized
instruction on-chain, with complete negative tests and verifiable public evidence. Until that milestone exists and is
independently evaluated, no claim of quantum-safe asset custody or blockchain-native post-quantum settlement should
be made.

THE QUANTUM QLI POSITION

Build useful operations now. Make signed intent independently inspectable. Advance toward on-chain enforcement through demonstrable programs and negative tests - without confusing an application check with a
consensus rule.

CHAPTER 18

Glossary and source register

Selected terminology

TERM MEANING

Attestation Seal WOTS/Merkle reference proof over a limited canonical set of launch fields.

Authorization intent Versioned signed representation of the requested operation and its exact review context.

Application gate Client-side rule checked before wallet approval; bypassable by other clients.

DBC Meteora Dynamic Bonding Curve pool and configuration workflow.

DAMM v2 Supported target for the current DBC pool migration setting.

Ed25519 Conventional signature scheme used by ordinary Solana wallet authorization.

WOTS leaf One-time signature slot; reuse can undermine security.

Nonce Unique challenge/replay guard; current app protection is session-local.

Program-enforced Rule checked during a Solana instruction execution, independent of QLi UI.

Verified on-chain Label reserved for confirmed transaction plus relevant re-read of account state.

Public GitHub sources (baseline)
[R1] Repository README. View source.

[R2] Original technical white paper (v0.1). View source.

[R3] Post-quantum algorithms and reference WOTS. View source.

[R4] Quantum Identity derivation and proof model. View source.

[R5] Launch Forge configuration and seal scope. View source.

[R6] Solana and Meteora protocol background. View source.

[R7] Security and custody boundaries. View source.

[R8] Public API description. View source.

[R9] Code, documents and brand licensing. View source.

Updated implementation evidence

[U1] Updated launch execution source and deployment checklist reviewed 11 October 2026.
[U2] Current Flywheel Live implementation,
tests and docs reviewed 11 October 2026. [U3] Quantum Authorization v1 code, verifier, integration and test report reviewed 11 October
2026. [U4] Current vault prototype and limitations docs. These newer implementation snapshots may not yet be mirrored in the public
GitHub main branch.

Readers should reconcile current code, deployed application behavior, RPC status and the public repository history before treating any
feature as production-ready. Source retrieval date: 11 October 2026.

Standards and technical context

NIST FIPS 203 - ML-KEM.

NIST FIPS 204 - ML-DSA.

NIST FIPS 205 - SLH-DSA.

BUILD ATTRIBUTION

Builder credit (project-provided): Nous Hermes Agent. This attribution does not establish that Nous Research
operates QLi infrastructure, co-developed a deployed verifier, formally partners with the project, or audited its
cryptography.

Copyright and rights in original artwork and marks remain with their respective holders.
This publication is a technical description and may be revised as
protocol implementations and independent evidence evolve.